

# Analyse des Design Patterns et Techniques - Blog PHP CFPC

---

Ce document recense les design patterns et techniques utilisés dans le projet PHP Blog CFPC.

## 1. Pattern Singleton (Singleton Pattern)

---

**Fichier:** database/database.php

```
function getPdo(): PDO
{
    static $pdo = null;
    if (null === $pdo) {
        $pdo = new PDO(...);
    }
    return $pdo;
}
```

**Technique:** Utilisation de `static` pour maintenir une seule instance de PDO throughout l'application. C'est un Singleton déguisé via une fonction.

## 2. Pattern Repository / Data Access Object (DAO)

---

**Fichiers:**

- app/functions/article.php
- app/functions/user.php
- app/functions/comment.php

Chaque fichier encapsule les opérations CRUD pour une entité spécifique. Ce sont des fonctions utilitaires qui agissent comme une couche d'accès aux données.

## 3. Pattern Service Layer

---

Les fichiers dans `app/functions/` servent de "services" qui contiennent la logique métier (count, find, create, delete).

## 4. Enum (PHP 8.1+)

---

**Fichier:** app/Enums/Role.php

```
enum Role: string
{
    case ADMIN = 'admin';
    case USER = 'user';

    public function label(): string { ... }
    public function isAdmin(): bool { ... }
}
```

**Pattern:** Type-safe Enum avec méthodes - remplace les constantes classiques.

## 5. Pattern Flash Messages (Notification/Feedback)

---

**Fichier:** flash.php

```
function flash_set(string $type, string $message): void
function flash_get(): ?array
```

**Pattern:** Utilise la session pour afficher des messages temporaires (success/error) qui disparaissent après lecture. C'est le pattern "Flash" connu dans Rails/Symfony.

## 6. Pattern Helper Functions / Utility

---

**Fichier:** app/helpers.php

Fonctions globales réutilisables:

- clean\_input() - Sanitization
- createSlug() - URL slugification
- checkAuth() / checkAdmin() - Auth middleware
- render() - Template rendering
- redirect() - Navigation

## 7. Pattern Template Engine / View

---

**Fichier:** app/helpers.php:75

```
function render(string $path, array $variables = [], string $layout = 'blog-layout')
{
    extract($variables);
    ob_start();
    require_once "resources/views/" . $path . "_html.php";
    $pageContent = ob_get_clean();
    require_once "resources/views/layouts/{$layout}/{$layout}_html.php";
}
```

**Technique:**

- Output buffering ( ob\_start/ob\_get\_clean )
- extract() pour rendre les variables disponibles dans la vue
- Layout system avec inheritance

## 8. Pattern Middleware / Guard

---

```
function checkAuth(): void  
function checkAdmin(): void
```

Vérification des permissions avant l'accès aux pages.

## 9. PDO Prepared Statements (Security)

---

Partout dans les fichiers de fonctions - utilisation de requêtes paramétrées pour éviter les injections SQL.

```
$query->bindValue(':id', $id, PDO::PARAM_INT);
```

## 10. Pagination Pattern

---

**Fichier:** index.php

```
$totalItems = countArticles();  
$totalPages = ceil($totalItems / $itemsPerPage);  
$offset = ($currentPage - 1) * $itemsPerPage;  
$articles = findAllArticles($itemsPerPage, $offset);
```

## 11. Subquery / Jointure dans SQL

---

**Fichier:** app/functions/article.php:18-21

```
SELECT articles.*,  
       (SELECT COUNT(*) FROM comments WHERE comments.article_id = articles.id) AS  
comment_count  
FROM articles
```

## 12. Role-Based Access Control (RBAC)

---

Utilisation de l'enum Role pour gérer les permissions admin/user dans login.php:47 :

```
if ($_SESSION['auth']['role'] === Role::ADMIN->value) {  
    redirect("admin.php");  
}
```

## 13. Separation of Concerns

---

Organisation du code:

- database/ - Connexion
- app/functions/ - Logique métier
- app/Enums/ - Types
- resources/views/ - Vues
- Fichiers root - Contrôleurs/Front Controllers

## 14. Security Techniques

---

| Technique                       | Implementation                     |
|---------------------------------|------------------------------------|
| <b>XSS Prevention</b>           | htmlspecialchars() , clean_input() |
| <b>SQL Injection Prevention</b> | Prepared statements ( bindValue )  |
| <b>CSRF</b>                     | Non implémenté (à ajouter)         |
| <b>Password Hashing</b>         | password_verify()                  |
| <b>Type Hinting</b>             | declare(strict_types=1)            |

## Résumé des Patterns

---

| Pattern          | Statut                 |
|------------------|------------------------|
| Singleton        | ✓ PDO connection       |
| Repository/DAO   | ✓ app/functions/*      |
| Service Layer    | ✓                      |
| Enum (PHP 8.1)   | ✓ Role.php             |
| Flash Messages   | ✓ flash.php            |
| Helper/Utility   | ✓ helpers.php          |
| Template Engine  | ✓ render()             |
| Middleware       | ✓ checkAuth/checkAdmin |
| Pagination       | ✓ index.php            |
| RBAC             | ✓ Role enum            |
| Output Buffering | ✓ ob_start()           |

## Suggestions d'amélioration

---

1. **AJAX endpoints** - Créer une API REST
2. **CSRF Protection** - Ajouter des tokens
3. **Form Validation** - Créer un composant de validation
4. **Dependency Injection** - Utiliser un container IoC
5. **Model Classes** - Créer des classes plutôt que des fonctions procédurales